Build, Test, Success: Verification in the Loop Turns LLM Agents into Brick Builders

Maksym Bondarenko
Independent Researcher
maksym.d.bondarenko@gmail.com

Abstract

Can a language model build a physical structure? We study this question with LEGO-style bricks, a domain where the parts are fixed, expert data is plentiful, and every claim a build makes—parts connect, nothing overlaps, the structure stands—can be checked by machine. We propose multi-step iterative brick generation with error recovery: a general frontier model builds inside an interactive loop where each placement is expressed in a connection language and checked as it is made, and a validator reports which parts are wrong and how, not just that the build failed. On a suite of thirteen tasks with exact automatic grading, direct one-shot generation passes two tasks—the two most repetitive shapes—while the same model inside the loop passes all thirteen, including a 684-part Taj Mahal. A five-condition ablation shows where the capability comes from: telling the model which parts are wrong recovers most of the tasks, building inside the environment recovers all but one, and verifying the finished build recovers the last. The environment, the tasks, the validator with its measured error, every recorded session, and the grade records behind every number in this paper are at `https://github.com/danish-bricks-builder/build-test-success`.

1 Introduction

Building with LEGO bricks is an iterative process: parts are placed one at a time, each placement must form a real connection, and the structure must remain stable as it grows. Can a language model perform this task? In this work, we study language-model agents that generate LEGO constructions over many steps and recover from their own errors along the way. We find that with the right support this works well, and that the decisive ingredient is verification: the same model that fails almost every construction task on its own completes nearly all of them inside our build-and-check loop (Fig. 1).

LEGO is an attractive domain for studying physical construction with machine learning. The set of parts is fixed and cannot be modified, which reduces the problem to assembly and, importantly, makes rigorous validation tractable: when the geometry of every part is known, every placement can be checked by machine. Large amounts of expert-designed data are available in the machine-readable LDraw format, including official models with step-by-step build sequences, whereas comparable data for general CAD remains scarce [Koch et al., 2019, Wu et al., 2021]. A valid brick assembly can be physically built immediately, so questions of manufacturability do not arise. At the same time, the domain retains considerable depth: with Technic parts, gears, and motors, brick constructions can approximate real mechanical systems (Fig. 2).

A brick model makes a set of physical claims: parts are connected, no two parts occupy the same space, and the structure stands under gravity. We observe that while current models perform poorly when asked to generate brick models directly—consistent with prior reports [Liu et al., 2026]—their failures are largely local and inexpensive to detect: a part that floats, a wall that extends one stud too

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: Evaluation of Interactive Agents.

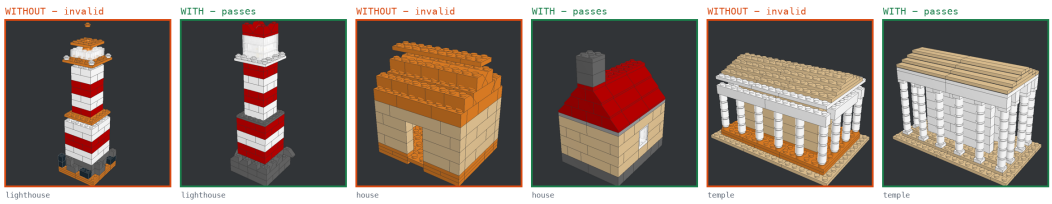


Figure 1: Three tasks without and with the validator in the loop. Without it, direct generation produces plausible-looking structures whose orange parts float or overlap; with it, the same tasks pass every check. The orange highlights are the validator's own output.

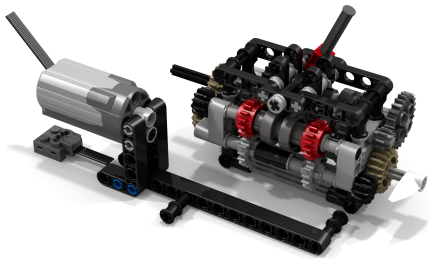


Figure 2: The ceiling of the domain: a motorized 6-speed gearbox built entirely from LEGO Technic parts — gears, clutches, driveshafts, and a gear lever. Brick constructions of this kind approximate real mechanical systems. Image: David Luders, CC BY-SA 2.0, via Wikimedia Commons.

far (Fig. 1, left). Prior work has addressed brick generation primarily by training specialized models on brick data [Pun et al., 2025, Kulits and Schmid, 2026]. We take a different approach: rather than training a model, we place a general frontier model in an interactive loop where its errors are caught and provided back to the model to be repaired.

To our knowledge, we are the first to propose multi-step iterative LEGO generation with error recovery. We enable it with three components. First, a *connection language*, in which a placement names a pre-verified seating of one part on another rather than a coordinate (§3.1). Second, an *interactive environment* that checks every placement at the moment it is made (§3.3). Third, a *validator* of connection validity, collision, and physical stability, whose reports identify the offending parts and the nature of each problem (§3.6). We demonstrate the faithfulness of our validator by running it on existing, human-made LEGO sets.

Our experiments show that the same frontier model, given the same prompts, passes two of thirteen construction tasks when generating brick files directly, and all of them when building inside the proposed loop with the validator at hand (§4). The ablation separates the components. A located report of what is wrong, pasted back as text, lifts direct generation to 10 of 13. The environment, refusing each invalid placement as it is made, reaches 12 of 13 with no verification of the finished build. Verification of the finished build closes the monumental tasks that remain. The environment, the validator, the task suite, every recorded session and the grade records are public at `https://github.com/danish-bricks-builder/build-test-success`.

2 Related Work

AI for LEGO. Prior work approaches brick[1] generation by training models on brick data. LegoGPT [Pun et al., 2025] fine-tunes a language model to emit placements token by token, undoing a placement whenever a physics check fails; StableLego [Liu et al., 2024] contributes stability analysis; earlier work assembles target shapes from identical unit bricks [Chung et al., 2021]. BrickComposer [Liu et al., 2026] benchmarks step-by-step assembly from instruction-manual images and finds zero-shot frontier models below 0.5% per-step success—independent confirmation of our raw baseline.

[1] LEGO® is a trademark of the LEGO Group, which does not sponsor, authorize or endorse this work.

BrickNet [Kulits and Schmid, 2026] is the closest work and the best evidence that our representation choice is right: it independently makes connectivity the core of the representation—a 320k-structure dataset with typed connector annotations, where builds are written as chains of attach statements rather than coordinates—and shows that models fine-tuned on this representation stay valid past 94 placements where coordinate output collapses within about 50. BrickNet then generates *freely*: its models still produce inter-part collisions, and its discussion calls for inference-time approaches to guide decoding. This paper builds that layer. Our placements are restricted to enumerated, collision-pruned seatings and checked as they are issued; a validator with located reports and a stability check sit on top; and the builder is a general frontier agent—we fine-tune no model, so every capability difference we report comes from the environment. Where BrickNet curates fan files at scale, we verify our language by re-expressing official sets in it and rebuilding them from the result (§3.2); where its format keeps continuous parameters (any rotation angle, any axle depth), our catalog trades that expressivity for soundness—illegal placements cannot be written—and we measure what the trade costs (§3.2). Gear semantics beyond our joint model appear in Xu et al. [2019].

Agents that build in virtual worlds. LTRON [Walsman et al., 2022] is the closest interactive environment: LDraw-based, with agents that disassemble and rebuild structures. It couples perception and action at the pixel level—the agent clicks connection points on rendered images—which bounds its tasks to a handful of bricks; we keep vision as observation and make connections the action space, and build at official-set scale. Voyager [Wang et al., 2023] shows open-ended tool-using agents in Minecraft, where placements are grid-snapped voxels; brick placements can be representable yet physically illegal, which is the gap our checking closes. BlenderGym [Gu et al., 2025] evaluates agents in a professional 3D tool.

Text-to-CAD and assembly. DeepCAD [Wu et al., 2021], ABC [Koch et al., 2019], the Fusion 360 Gallery line [Willis et al., 2021, 2022], and AutoMate [Jones et al., 2021] study CAD generation and mating for CAD assemblies. They document the scarcity of high-quality data as a major blocker, which is why we chose to focus on LEGO bricks.

Evaluating interactive agents. Benchmarks for tool-using agents grade the outcome of a long interaction rather than a single response: SWE-bench [Jimenez et al., 2024] judges whether the repository's tests pass after the agent's patch, WebArena [Zhou et al., 2023] and OSWorld [Xie et al., 2024] judge the resulting page or system state, and BlenderGym [Gu et al., 2025] judges the edited 3D scene. We follow the same principle—the final model file is graded by code that never sees the conversation—and add three elements that this setting makes possible. First, the graders are part of the environment: the same checks that grade the final file also gate the session when the agent declares completion and write the repair messages it reads, so the evaluation instrument is itself part of what is being evaluated. Second, the checkers report *where* a build fails and in what way, not only that it fails, which is what makes bounded repair meaningful to measure. Third, to make sure the graders we build do not produce false positives or false negatives, we run them on existing human-built LEGO sets, which we know to be physically valid, and report their error (§3.2, §3.6.3). Every session is recorded with a model snapshot after each action and can be replayed step by step, which we found essential for attributing failures to the model, the environment, or the graders.

Existing LEGO software. LDraw[2] defines the format and part library; LeoCAD[3] and BrickLink Studio[4] are editors for people; LDCad[5] pioneered machine-readable connector annotations. Although this software allows building with LEGO bricks, and some of it can be scripted, none provides an API through which an AI agent can place bricks and check placements, so we built our own environment.

[2] `https://www.ldraw.org`
[3] `https://www.leocad.org`
[4] `https://www.bricklink.com/v3/studio/download.page`
[5] `http://www.melkert.net/LDCad`

3 Methodology

3.1 Representing LEGO: LDraw and the connection language

The industry standard for digital brick models is LDraw[6]. A part library gives a 3D mesh per part id, and a model file places parts one line at a time: part id, absolute position, a 3x3 rotation matrix (Fig. 3a, left). This format is convenient for rendering but poorly suited to building: a builder does not reason in coordinates, and neither does an instruction booklet. Both reason about what connects to what, and in a valid build the positions follow from the connections.

There is no accepted machine language for brick connections, so we built one. The first part of a build is an *anchor*, placed in absolute space. Every further part is an *attach* statement: this part, onto that placed part, in seating k (Fig. 3a, middle). The seating index refers to a catalog we compute ahead of time: for every pair of part types, every legal way the two can join — each stud offset, each rotation, each pin and axle insertion — with the colliding ones already pruned. Figure 3 shows one such catalog in full: even for the simplest pair of bricks, the set of legal joinings is rich — offsets, rotations, and underside attachments — yet finite and known in advance. One statement therefore names a complete placement that is known to be legal before it is ever used. The language provides no way to write a floating or overlapping placement.

3.2 Building and validating the language

The connection language rests on two computed artifacts. The first is a *connector library*: the LDraw part library provides geometry for 18,386 part types but says nothing about how parts join. We take the connectors from the LDCad Shadow Library [Melkert, 2026], which annotates each part with its studs and the sockets that receive them, pins and pin holes, axles, bars, and clips—each with a position, an axis, a gender, and a cross-section profile—and expand its grid specifications into individual connectors; the mating tests and the joint rules below are ours. (BrickNet [Kulits and Schmid, 2026] annotated the same library independently and concurrently.) Two connectors mate when four geometric tests pass: their axes are parallel, they are concentric within tolerance, their spans overlap deeply enough to be seated, and insertion enters through an open mouth rather than a blind face. The connections found between two parts are then judged by an ordered list of rules that decides whether the pair is rigid or forms a moving joint. The second artifact is the *seating catalog* of §3.1, enumerated from the connector library ahead of time.

We validate the language against reality with a *decompiler*, which re-expresses an arbitrary LDraw file as a program:

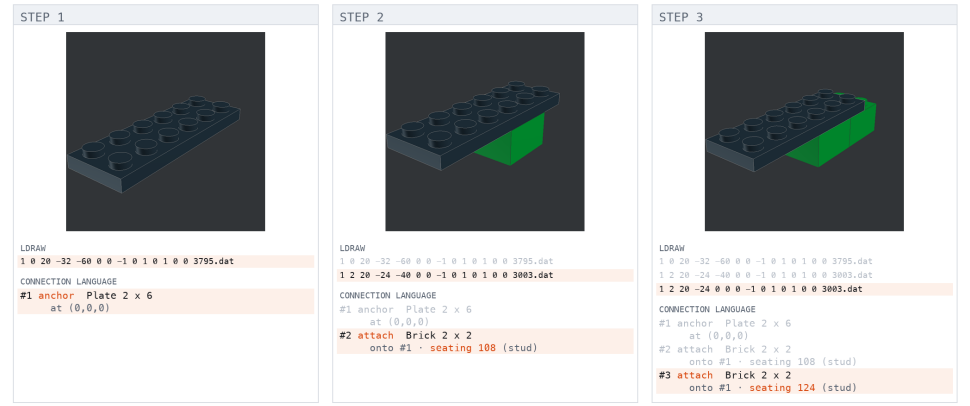
```
for each part P in build order:
    earlier <- connected partners of P placed before it
    if none: emit ANCHOR(P)
    parent <- the partner P grips hardest (most contacts)
    k <- match(pose of P relative to parent, catalog[pair])
    if k:  emit ATTACH(P, onto=parent, seating=k)
    else:  emit CONTACT(P, onto=parent)     # fallback outside the language
```

Applied to a corpus of official models (1,464 sets in LDraw format with authored build sequences), the decompiler re-derives 91% of placements as catalog seatings. The remaining 9% are placements the catalog cannot express—axles inserted to depths between the sampled ones, and plates rotated on a single stud to off-grid angles—the cost of making illegal placements unwritable. Every emitted program is certified by *replay*: it is executed from an empty workspace and the result is compared against the original file. The corpus thus serves as ground truth that the language can express how real sets are actually built.

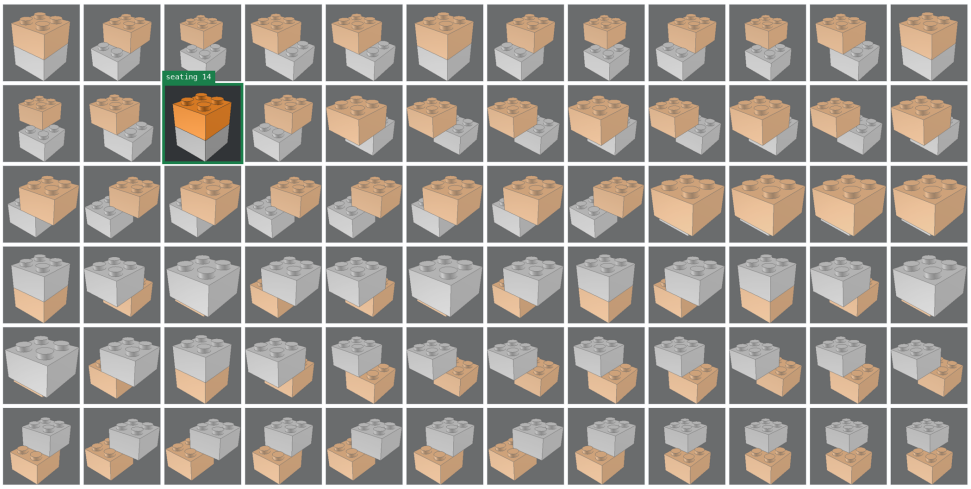
3.3 The interactive environment

The agent builds through a multi-turn tool interface. The parts placed so far in the session constitute the state, which persists across turns. The mutating actions are the connection language—anchor, attach, remove—plus a *settle* operation that lets an unsupported part drop until it rests on the structure below; the non-mutating actions are lookups: search the part catalog, list the legal seatings at a

[6] `https://www.ldraw.org`



(a) three build steps in both encodings



all 72 legal seatings of a 2x2 brick on a 2x2 brick — an attach statement names exactly one of them

(b) a seating catalog: all 72 legal joinings of one pair of parts

Figure 3: The connection language. (a) Three build steps in both encodings: absolute LDraw lines and attach statements naming a seating. (b) A seating catalog: all 72 legal joinings of a 2×2 brick onto another, computed ahead of time with colliding placements pruned; a statement selects one entry (green). Placements outside the catalog cannot be written.

location, inspect the current build, render a view, and sketch a layout of named subsystems before building. The central property of the environment is that every placement is checked at the moment it is made: an attach that would form no real connection, or would overlap an existing part, is rejected with a reason and never enters the build. Every session records each action together with a snapshot of the model after it, which makes all of our experiments replayable.

3.4 The build–test–recover loop

The validator of §3.6 participates in the session in two ways. First, as tools: the agent may invoke validity and stability checks at any point during the build. Second, as *gates*: when the agent declares the build complete, the validator runs in order—connection validity, then stability, then the task's size requirements—and a failure does not end the session. Instead it opens a bounded repair round (two per gate) whose message contains the diagnosis: which parts float, which rows of studs protrude past the allowed footprint, in which direction a substructure would fall. The agent then edits the build and the gate runs again.

Every session is recorded action by action with a model snapshot after each step, so any session can be replayed and inspected.

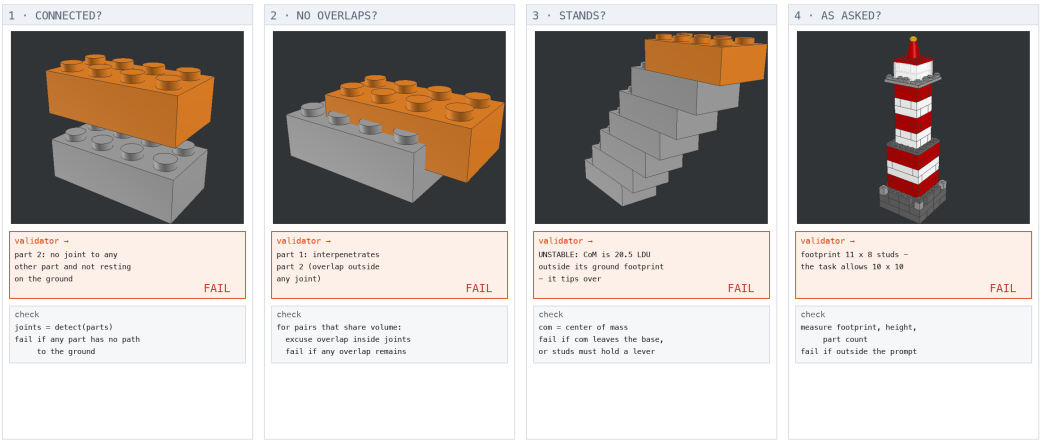


Figure 4: The four grading questions and example failures, with the validator's message for each. The same text drives repair rounds.

3.5 Tasks

Our tasks are single prompts with machine-checkable requirements. The prompt is given once; no human intervenes afterwards. The main family is *construction*, in three size bands that rise monotonically: small structures (25–50 parts minimum: a wall, a tower, a house, a stepped pyramid), medium landmarks (70–140: a lighthouse, an aqueduct, a temple, a pyramid, a colosseum), and monumental builds (400 parts minimum: Notre-Dame de Paris and the Taj Mahal). One *editing* task loads an official set and asks the agent to raise it onto a pedestal; at least 90% of the original parts must survive, measured as described below. One *restricted inventory* task asks for a dinosaur using only the parts of one official set; the environment rejects any placement once a part runs out. Thirteen prompts in total; the full prompts ship with the code release.

3.6 Evaluation

A *session* ends when the agent declares completion and clears whatever gates its condition includes, or exhausts its budget. We then grade the final model file offline, with code that is blind to everything the agent said during the session, so the agent's own claims of success play no role. Grading asks four questions in turn (Fig. 4); a build passes only when the answer to all four is yes, and the first "no" is the mark the results table shows: $\times^{\mathrm{v}}$ invalid (a part floats or overlaps another), $\times^{\mathrm{s}}$ falls over, $\times^{\mathrm{f}}$ too big for the footprint, $\times^{\mathrm{r}}$ another requirement missed.

3.6.1 Connection validity

A build is connection-valid when every part is held by real connections: the mate detector of §3.2 runs over the final model, and every part must reach the ground through real joints or rest on it; a part joined only to parts that never reach the ground floats with them. The report names each unattached part.

3.6.2 Collision

Parts are compared at the level of their true volumes, including internal cavities. Connected parts are supposed to overlap where they join—a stud fills the tube it sits in—so overlap is excused exactly inside the engagement zone of a detected connection and counted everywhere else. Unconnected pairs collide on any overlap beyond a small noise tolerance; connected pairs collide only on overlap outside the engagement zones of their joints.

3.6.3 Stability

We use simplified rigid-body statics in the spirit of Pun et al. [2025], Liu et al. [2024]. Part masses come from their volumes at the density of ABS plastic:

```
for each connected structure S:
    com <- center of mass of S
    if com projects outside S's ground-contact area:
        report(S, "topples", direction)
    for each substructure H held only through stud connections:
        if the moment of H's weight at the stud interface
           exceeds the stud clutch limit:
            report(H, "breaks off", direction)
```

The clutch limit is the largest moment a stud connection can resist before the parts separate, taken from published measurements of stud friction. We calibrated this checker on official sets: 87% pass, and the failures concentrate in posed display scenes (vehicles on stands) rather than the building-class structures used in our tasks.

3.6.4 Task requirements

Finally we check that the build does what the prompt asked. For construction tasks this means the part-count range, the minimum height, and the footprint, measured from each part's box turned the way the part is placed. For editing tasks we measure how much of the original set survived: we find the single translation that best maps the original onto the result and count the parts that match under it, so that moving the original as one rigid piece scores perfectly while dismantling it does not. For inventory tasks the final bill of materials must be a subset of the allowed parts. A task **passes** when the build is connection-valid, collision-free, stable, and meets every requirement. Everything else about a session—the action sequence, rejections, timing, the part count over time—is recorded but not scored. Our checks bound size and physics, not meaning: no grader verifies that a temple has columns; only the rendered figures show that.

3.7 Experimental setup

We evaluate one frontier model (Claude Opus 5, `claude-opus-5`) with fixed prompts under five conditions: every task under every condition, one session per cell. The prompt is the same in all five, the task text followed by its machine-checked limits (footprint, minimum height, minimum part count). The conditions form a ladder; each adds one thing to the one before it.

one-shot: the model writes the whole LDraw file in one reply, with no environment. The reply may run to the model's output ceiling of 128k tokens, so even a monumental file fits. **one-shot+feedback**: up to four such replies. Between them the validator's report on the previous file is pasted back as text: the summary, then each faulty part by index, part number, colour, its line in the file, and the reason. The model answers with the complete corrected file. **interactive**: the model builds inside the environment, placing parts in the connection language through tool calls. Each placement is checked as it is made and refused if it does not mate, but the finished build is never verified; the session ends when the agent declares it done. **interactive+final check**: as interactive, and when the agent declares completion the validator runs over the build. Each failing check reports where the problem is, and the agent gets two repair rounds per check. **interactive+check tools**: as interactive+final check, and the validator is also offered as tools the agent may call at any point during the build.

The first pair isolates located textual feedback; one-shot+feedback against interactive isolates the environment and its language; the last three isolate verification of the finished build and where in the session it happens. All conditions are graded identically and offline by the same code (§3.6). The editing task is given its starting set as part lines in the one-shot prompt and loaded into the environment in the interactive conditions.

We do not hold the conditions to a fixed budget: the question is the ceiling each approach reaches, not the cost of reaching it. One-shot gets one reply, one-shot+feedback four; the interactive conditions run until the agent declares the build done, within the environment's cap of 90 tool-loop iterations, from one minute to just under two hours. Every session is recorded and released.

Changes from the reviewed version. Every cell was rerun for this version with the released code, after three corrections. The footprint is now measured with each part's rotation; the reviewed grader took every part at its nominal width and failed builds that were within their limit. The report pasted back in one-shot+feedback now names the faulty parts; the reviewed version gave only their counts.

The interactive prompt now states the task's limits, as the one-shot prompt always did, and sessions are no longer cut at a fixed time. The ablation, previously on a seven-task subset, now covers every task, and the editing task is attempted one-shot.

4 Results

Table 1 is the whole experiment: thirteen tasks under five conditions, one session per cell, every cell graded by the same offline code. Figure 6 in the appendix shows every one of the sixty-five builds.

task	one-shot	one-shot +feedback	interactive	interactive +final check	interactive +check tools
small					
wall-doorway	✓	✓	✓	✓	✓
tower-square	$\times^{v}$	✓	✓	✓	✓
house-gable	$\times^{v}$	$\times^{v}$	✓	✓	✓
pyramid-steps	✓	✓	✓	✓	✓
medium					
lighthouse-tall	$\times^{v}$	✓	✓	✓	✓
roman-aqueduct	$\times^{v}$	✓	✓	✓	✓
greek-temple	$\times^{v}$	✓	✓	✓	✓
pyramid-giza	$\times^{v}$	✓	✓	✓	✓
colosseum-ring	$\times^{v}$	✓	✓	✓	✓
monumental					
notre-dame-de-paris	$\times^{v}$	$\times^{v}$	✓	$\times^{v}$	✓
taj-mahal	$\times^{v}$	$\times^{v}$	$\times^{v}$	✓	✓
edit					
edit-ghost-pedestal	$\times^{v}$	✓	✓	✓	✓
inventory					
inv-dinosaur	$\times^{v}$	✓	✓	✓	✓
pass	2/13	10/13	12/13	12/13	13/13

Table 1: Every task under every condition. ✓ every check passes; $\times^{v}$ invalid (parts float or overlap); $\times^{s}$ falls over; $\times^{f}$ too big for the footprint; $\times^{r}$ another requirement missed.

4.1 The two ends

One-shot generation passes two of the thirteen tasks, the plain wall and the stepped pyramid, the two shapes in which one correct course repeats. Every other one-shot file is a plausible shape whose parts float or overlap (Fig. 1). The same model with the environment and the validator at hand passes all thirteen, including a 684-part Taj Mahal and an 875-part Notre-Dame.

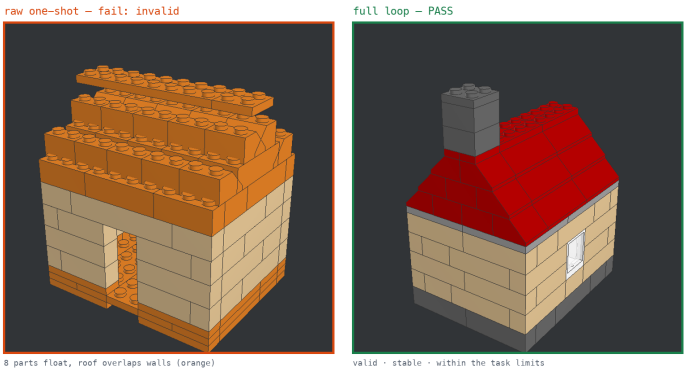


Figure 5: The house task under the two headline conditions. Left: raw one-shot generation—a plausible shape whose orange parts float or overlap the roof. Right: the same model inside the full loop passes every check. Borders and labels are the offline graders' verdicts.

4.2 What each step of the ladder buys

A located report does most of the work. Told which parts are wrong and why, the one-shot model repairs its own file: 10 of 13 pass within four replies, among them the colosseum, the temple, the editing task and the inventory task. What it cannot do is rebuild a file of eight hundred parts whose faults are structural. The three failures are the house, which stays invalid across four replies, and the two monumental tasks, where hundreds of parts float in the first reply and the report names forty of them at a time; Notre-Dame still ends four invalid parts short of passing.

The environment provides validity. Building piece by piece, with each placement checked as it is made, 12 of 13 tasks pass with no verification of the finished build at all: invalid geometry cannot accumulate, and the agent, told the limits, keeps to them. The one failure is the Taj Mahal, a single collision among 671 parts that nothing in this condition asks the agent to look for.

Verification of the finished build closes the rest. With the validator run at completion and two repair rounds, the Taj Mahal passes and Notre-Dame does not: when its repair rounds run out the build still has parts out of place and is wider than allowed. With the validator also available as tools during the build, every task passes. The two conditions differ by that one cell, so what the tools add is the chance to check before the end rather than only at it, which matters exactly for the largest build.

5 Discussion

In this domain, capability is a property of the agent and what it is given. A located report of what is wrong turns a model that fails almost every task into one that passes most of the suite; an environment that refuses an invalid placement does the rest for all but the largest builds; verification of the finished build separates most from all. We believe the shape of the domain explains why verification helps so much here: the claims a build makes are cheap to check by machine, the failures are local, and the repairs are small. Domains with the same shape should expect similar gains; domains where errors are global or checking is expensive should not.

This paper measures one slice of a larger space. Inputs can grow harder, from text alone to a reference image, and eventually to an instruction booklet followed to the finished set. Builds can grow harder, toward more connection variety, moving Technic mechanisms, and larger scale. Constraints can grow harder, toward building from a fixed inventory of parts, which our dinosaur task begins to probe. The environment is built to reach all of these.

For evaluating interactive agents we draw three practices from this work: grade the final artifact rather than the transcript; make checkers report *where* a build fails, because a located complaint is something an agent can act on; and record sessions in replayable form, because an outcome table cannot distinguish a clean pass from a demolition and rebuild.

The environment did not come for free. An early tool that fetched reference models for the agent to imitate failed in practice and was replaced by a human-designed planning tool; building environments for domains like this is currently human work, and we think that construction is itself the research. **Limitations.** One model family; one session per cell, so we measure the existence of the effect rather than its variance; statics rather than simulation; leave-one-out knockouts of individual components (the renderer, the diagnosis text, the repair budget) are defined but not yet run; and the grader error measured in §3.2 and §3.6.

6 Conclusion

We introduced a connection language for brick construction, an interactive environment that checks every placement as it is made, and a validator whose reports name the failing parts and the nature of each problem. With these components, a fixed frontier model goes from passing two of thirteen construction tasks to passing all of them. The ablation shows where the capability lives: a located report alone recovers most of the suite, the environment's checked placements recover all but one task, and verification of the finished build recovers the last. The environment, the tasks, the validator with its measured error, every recorded session, and the grade records are public at `https://github.com/danish-bricks-builder/build-test-success`.

References

Hyunsoo Chung, Jungtaek Kim, Boris Knyazev, Jinhwi Lee, Graham W. Taylor, Jaesik Park, and Minsu Cho. Brick-by-Brick: Combinatorial construction with deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Yunqi Gu, Ian Huang, Jihyeon Je, Guandao Yang, and Leonidas Guibas. BlenderGym: Benchmarking foundational model systems for graphics editing. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. arXiv:2504.01786.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

Benjamin Jones, Dalton Hildreth, Duowen Chen, Ilya Baran, Vladimir G. Kim, and Adriana Schulz. AutoMate: A dataset and learning approach for automatic mating of CAD assemblies. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 40(6), 2021.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. ABC: A big CAD model dataset for geometric deep learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Peter Kulits and Cordelia Schmid. BrickNet: Graph-backed generative brick assembly. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2604.22984.

Jiateng Liu, Bingxuan Li, Zhenhailong Wang, Rushi Wang, Kaiwen Hong, Cheng Qian, Jiayu Liu, Denghui Zhang, Katherine Driggs-Campbell, Manling Li, and Heng Ji. Brick-Composer: Using MLLMs for assembly with diverse bricks. arXiv:2606.05445, 2026.

Ruixuan Liu, Kangle Deng, Ziwei Wang, and Changliu Liu. StableLego: Stability analysis of block stacking assembly. *IEEE Robotics and Automation Letters*, 9(11):9383–9390, 2024.

Roland Melkert. LDCad shadow library: connection annotations for the LDraw part library. `http://www.melkert.net/LDCad/tech/shadowLib`, 2026. CC BY-SA 4.0.

Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. Generating physically stable and buildable LEGO designs from text. *arXiv preprint arXiv:2505.05469*, 2025.

Aaron Walsman, Muru Zhang, Klemen Kotar, Karthik Desingh, Ali Farhadi, and Dieter Fox. Break and make: Interactive structural understanding using LEGO bricks. In *European Conference on Computer Vision (ECCV)*, 2022.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 Gallery: A dataset and environment for programmatic CAD construction from human design sequences. *ACM Transactions on Graphics (SIGGRAPH)*, 40(4), 2021.

Karl D. D. Willis, Pradeep Kumar Jayaraman, Hang Chu, Yunsheng Tian, Yifei Li, Daniele Grandi, Aditya Sanghi, Linh Tran, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. JoinABLe: Learning bottom-up assembly of parametric CAD joints. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Rundi Wu, Chang Xiao, and Changxi Zheng. DeepCAD: A deep generative network for computer-aided design models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

Hao Xu, Ka-Hei Hui, Chi-Wing Fu, and Hao Zhang. Computational LEGO Technic design. *ACM Transactions on Graphics (SIGGRAPH Asia)*, 38(6), 2019.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A Every build



Figure 6: Every cell of Table 1, part one: the small tasks and the first three medium tasks. One row per task, one column per condition, the final build of each session rendered from one camera at one scale. A green frame: every check passes. A red frame: the file is not a valid build, parts float or overlap. An amber frame would mark a valid build that misses a requirement; there is none.

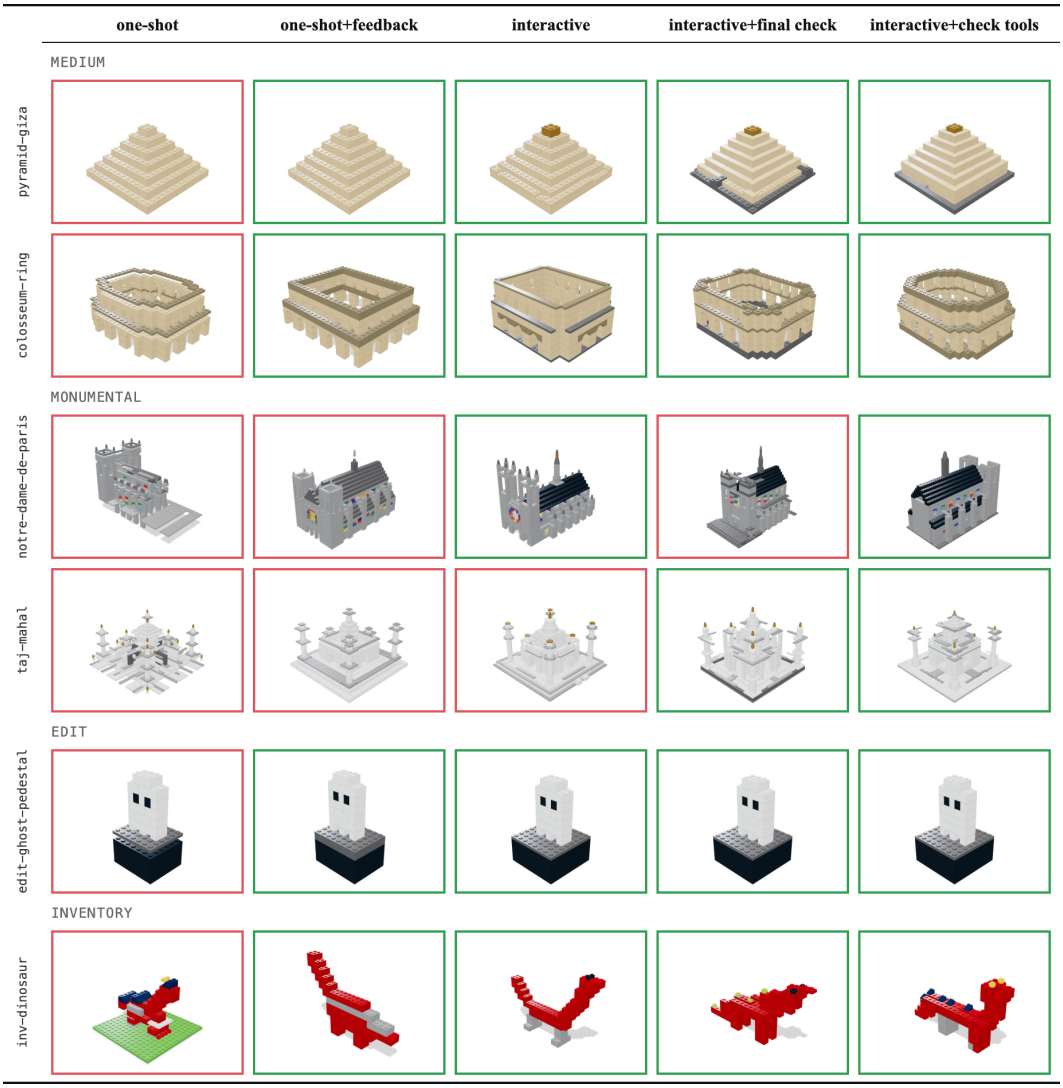


Figure 7: Every cell of Table 1, part two: the remaining medium tasks, the monumental tasks, the editing task and the inventory task. Frames as in Figure 6.